

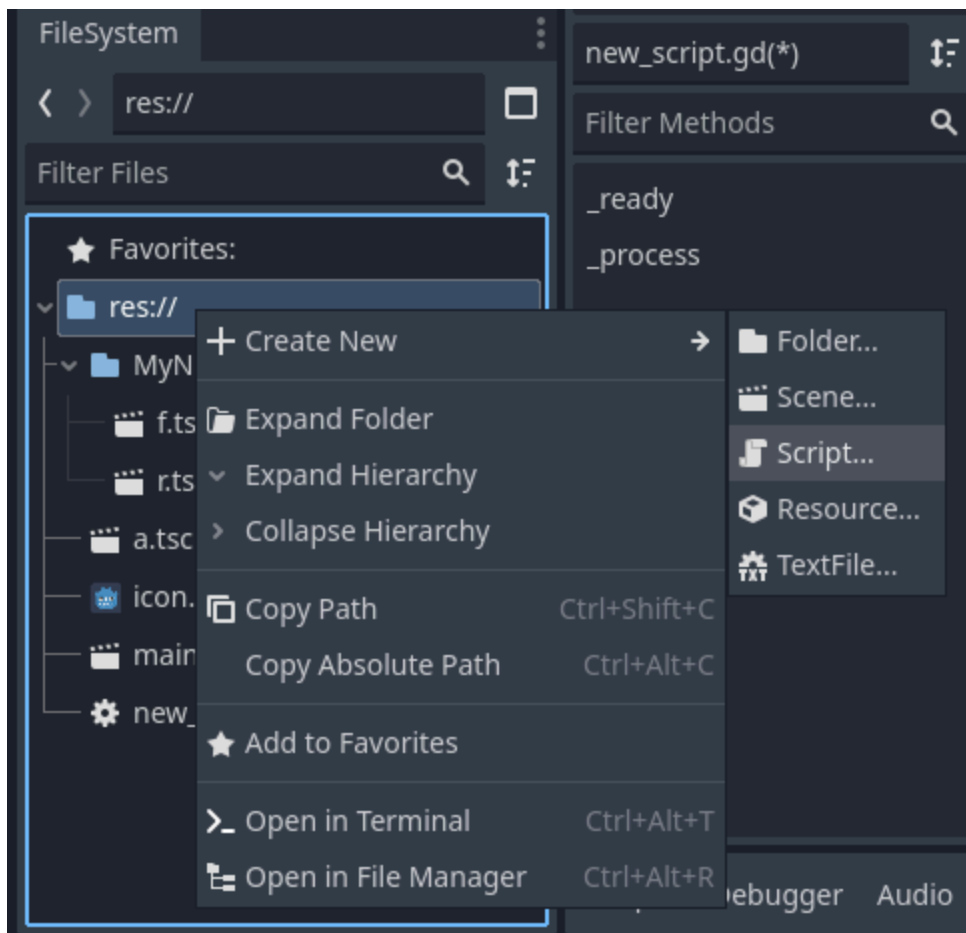
Basket Catch - Globals

So far we have only needed to access information from a single script from within itself. However, often we may need to share information between scenes and scripts. We can do this through the use of a special script called an *Autoload* or *Singleton*. This is a script that loads when our game is launched and can be accessed from anywhere within our game.

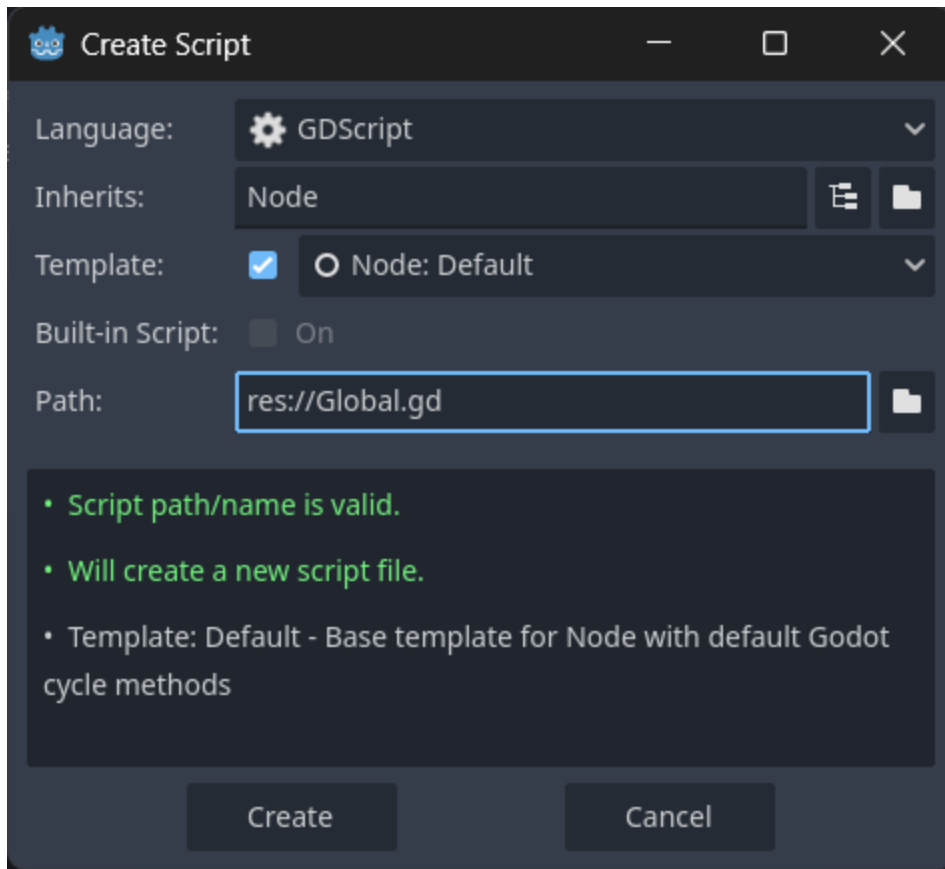
Making an Autoload Script

Unlike other scripts, Autoloads are attached directly to a node. Instead, we create the script file in the FileSystem and then set it as an Autoload in the *Project Settings*. Let's step through this process.

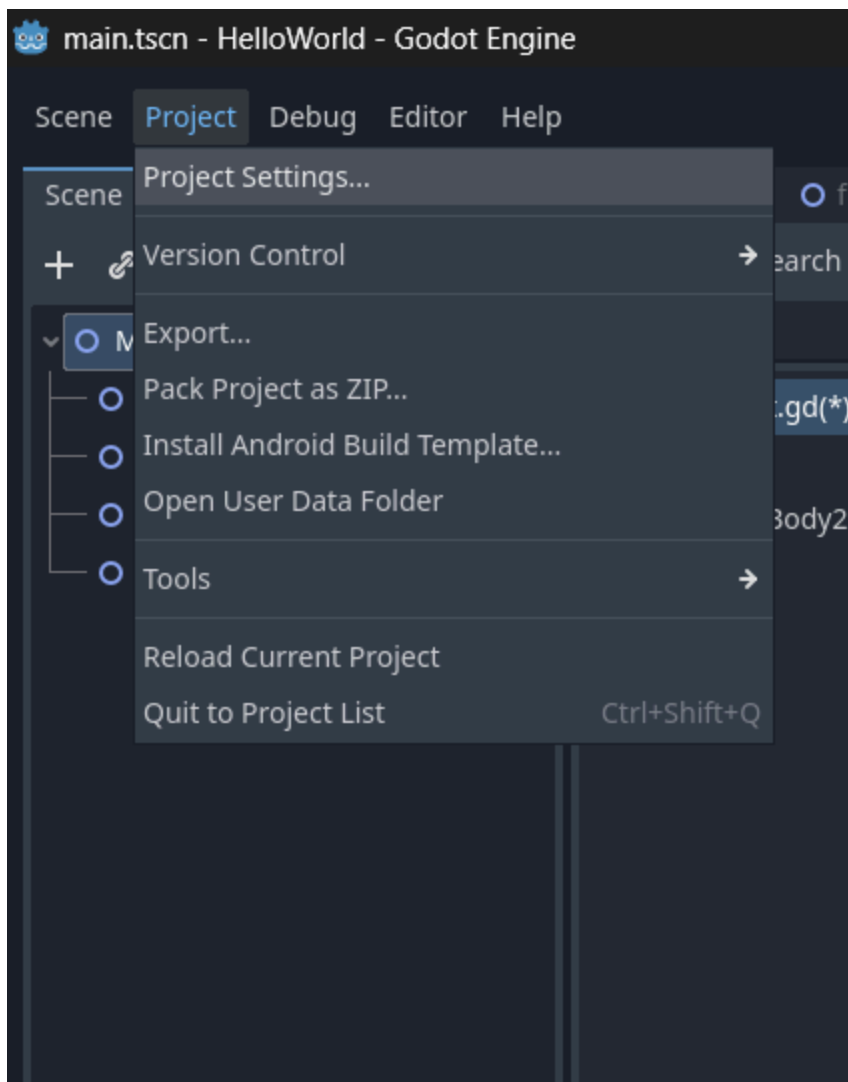
First, we'll create a new script by right-clicking on "res://" in the FileSystem, selecting "create new", and then selecting script:



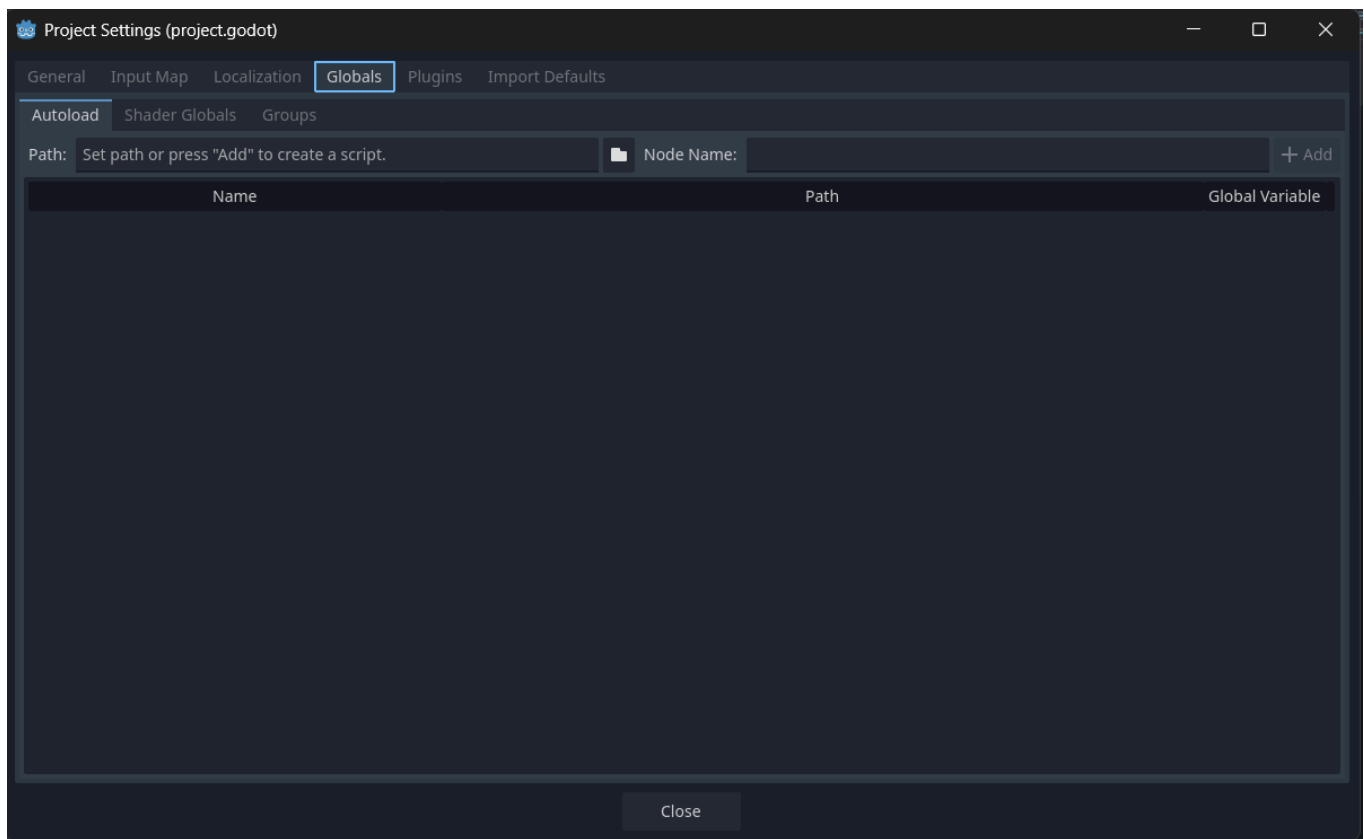
We will then name this script something descriptive. Often something like "Global" is a good choice. Make sure that when you name an *Autoload* script that we use a capital letter to start it:



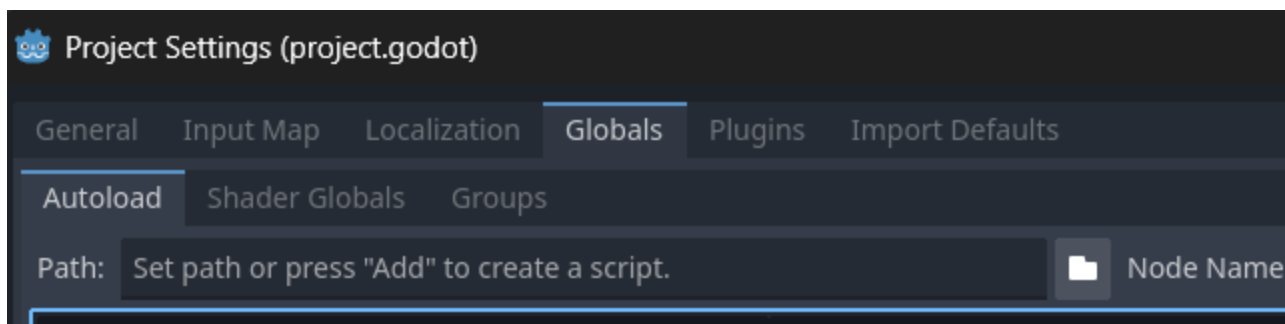
Next, we need to add it as an *Autoload* in the ProjectSettings. This can be found above the SceneTree dock:

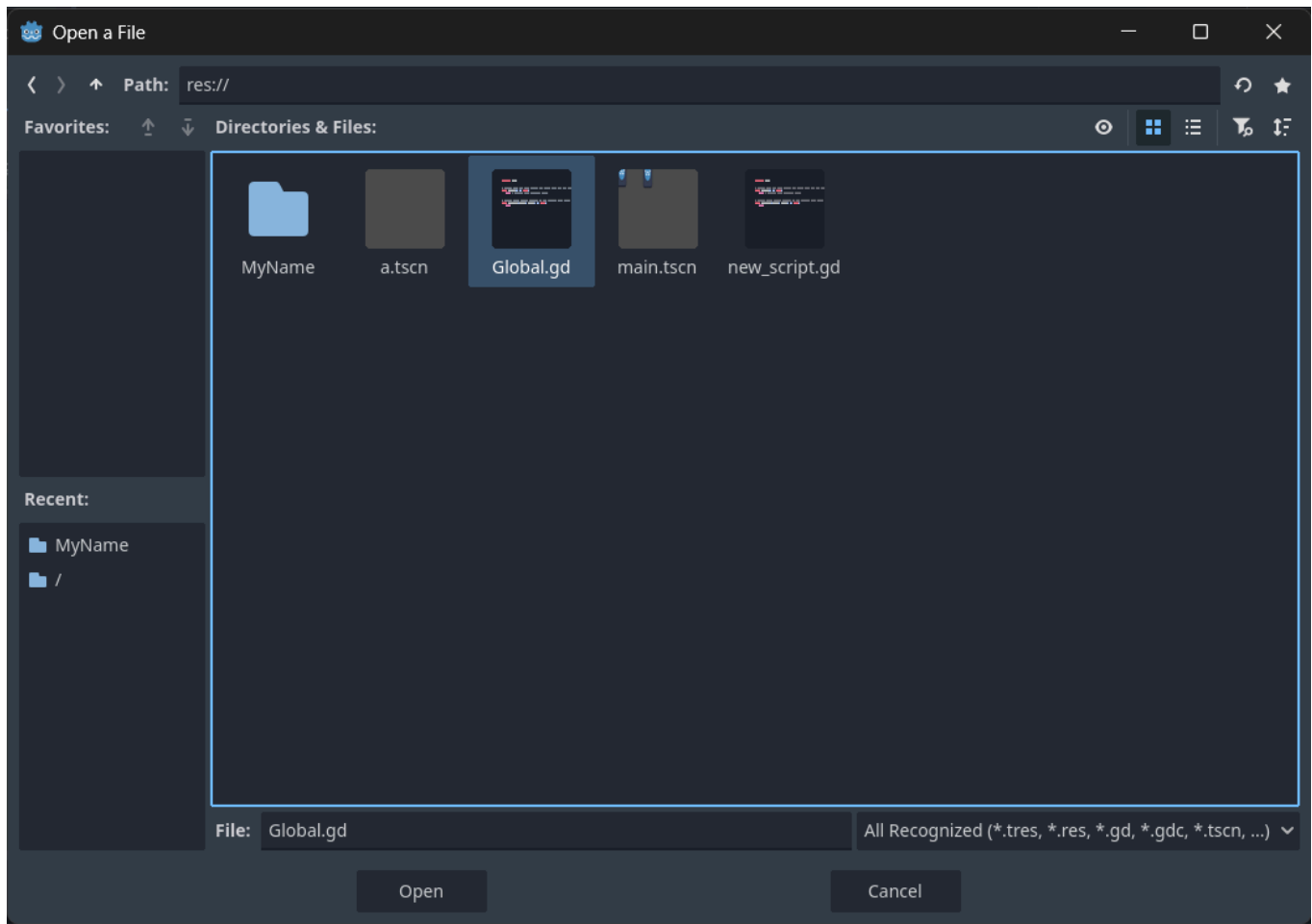


In our ProjectSettings we will go to the "Globals" tab:

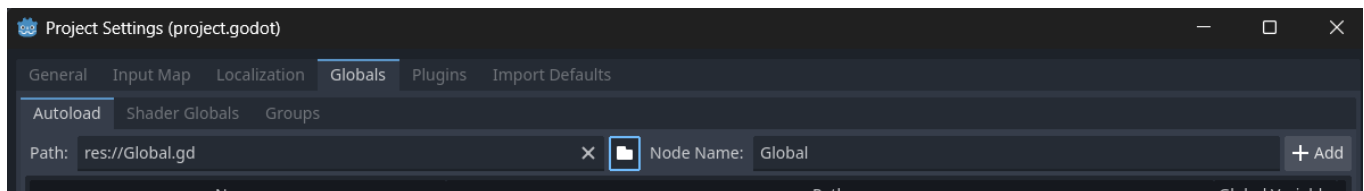


From here, we can select a script to load as an Autoload by clicking the folder icon next to the "Path" text field. This will open up a dialogue that lets us select the script we would like to use. Once you have chosen your desired script click "Open":

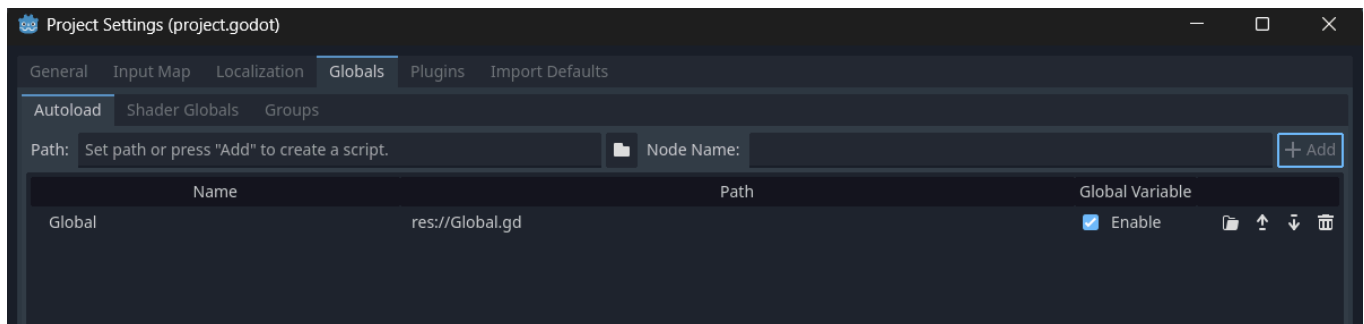




Now that we have selected our script, we can add it as an Autoload by clicking the "+ Add" button:



Great! We've successfully added a new Autoload script! Take note of the "Name" of the Autoload (in this case "Global"), we'll need it for later.



Using an Autoload in Code

Now that we've created our Autoload script we'll want to add some things to it. Often we will use an Autoload to both store *data* as well as perform *logic*. Does this sound familiar? It should, these are the same things a Class does! Just like a Class our Autoload can have *properties* and *methods()*. However, unlike the other pre-defined Classes in Godot we will need to define these properties and methods() ourselves.

Adding a property to the Autoload

Adding a new property to our Autoload script is quite simple. First we'll open the script by double clicking it in the FileSystem. Your script should look something like this:

```
extends Node

# Called when the node enters the scene tree for the first time.
func _ready() -> void:
    pass # Replace with function body.

# Called every frame. 'delta' is the elapsed time since the previous frame.
func _process(delta: float) -> void:
    pass
```

or this:

```
extends Node
```

Depending on if you selected a template or not.

We can add a *property* to our Autoload by declaring a *script-wide variable*. Let's consider that we may want to store our Player's health data on our Autoload. To create this player health property all we need to do is declare a variable:

```
extends Node

var player_health = 100

# Called when the node enters the scene tree for the first time.
func _ready() -> void:
    pass # Replace with function body.
```

```
# Called every frame. 'delta' is the elapsed time since the previous frame.  
func _process(delta: float) -> void:  
    pass
```

We can create as many properties as we would like this way! Let's add another for our player's lives:

```
extends Node  
  
var player_health = 100  
var player_lives = 3  
  
# Called when the node enters the scene tree for the first time.  
func _ready() -> void:  
    pass # Replace with function body.  
  
# Called every frame. 'delta' is the elapsed time since the previous frame.  
func _process(delta: float) -> void:  
    pass
```

Notice that we have not written any code to change these properties in our Autoload. We'll talk about accessing these properties from a different script later in this section.

Adding a method() to the Autoload

We can add a new method() to an Autoload by declaring a new *function*. Remember, a function is just a reusable block of code. This can be a very powerful tool if we are wanting to reuse complex logic in many places in our code. Let's make a method() that heals the player. We'll need to make sure that we don't heal the player for more than 100, and we'll want to add an *argument* to our function that allows us to choose how much the player is healed for:

```
extends Node  
  
var player_health = 100  
var player_lives = 3  
  
# Called when the node enters the scene tree for the first time.  
func _ready() -> void:  
    pass # Replace with function body.
```

```

# Called every frame. 'delta' is the elapsed time since the previous frame.
func _process(delta: float) -> void:
    pass

# Our new healing method()
func heal(amout) -> void:
    #Check if the player_health is less than 100
    if player_health < 100:
        player_health += amount

    #Checks if the player_health has been raised over 100, if it has set it to
100
    if player_health > 100:
        player_health = 100

```

This is a very efficient way to write code that is complicated. Imagine if we would need to rewrite the code in the heal() method() every place that it was used. We would have a much higher chance of making a mistake, and if we needed to change something in the code we would need to make sure to change it every place that it appears. Let's talk about how we can use these properties and methods() in other scripts!

Accessing an Autoload

Accessing a property

We can access an Autoload by calling its *Name* in any other script. Our Autoload's name in this case is "Global". Let's consider that we may want to access the "player_health" property from the Player's script itself. We can do this through dot notation:

```

extends CharacterBody2D

const SPEED = 300.0
const JUMP_VELOCITY = -400.0

# Called when the node enters the scene tree for the first time.
func _ready() -> void:

    # Prints the player's current health whenever the player scene loads
    print(Global.player_health)

    pass # Replace with function body.

```

```

func _physics_process(delta: float) -> void:
    # Add the gravity.
    if not is_on_floor():
        velocity += get_gravity() * delta

    # Handle jump.
    if Input.is_action_just_pressed("ui_accept") and is_on_floor():
        velocity.y = JUMP_VELOCITY

    # Get the input direction and handle the movement/deceleration.
    # As good practice, you should replace UI actions with custom gameplay
    actions.
    var direction := Input.get_axis("ui_left", "ui_right")
    if direction:
        velocity.x = direction * SPEED
    else:
        velocity.x = move_toward(velocity.x, 0, SPEED)

    move_and_slide()

```

Just like properties of other Objects, we can assign new values to the Autoload's properties. For example, we may have an Area2D attached to our player that we've connected the `body_entered` signal from. We may want to check when a PhysicsBody in the "enemy" group enters the Area2D and reduce the player's health by 10. Here's what that may look like:

```

extends CharacterBody2D

const SPEED = 300.0
const JUMP_VELOCITY = -400.0

# Called when the node enters the scene tree for the first time.
func _ready() -> void:

    # Prints the player's current health whenever the player scene loads
    print(Global.player_health)

    pass # Replace with function body.

func _physics_process(delta: float) -> void:
    # Add the gravity.
    if not is_on_floor():
        velocity += get_gravity() * delta

```

```

# Handle jump.
if Input.is_action_just_pressed("ui_accept") and is_on_floor():
    velocity.y = JUMP_VELOCITY

# Get the input direction and handle the movement/deceleration.
# As good practice, you should replace UI actions with custom gameplay
actions.
var direction := Input.get_axis("ui_left", "ui_right")
if direction:
    velocity.x = direction * SPEED
else:
    velocity.x = move_toward(velocity.x, 0, SPEED)

move_and_slide()

func _on_area_2d_body_entered(body: Node2D) -> void:
    # First, check if the entering PhysicsBody is in the "enemy" group
    if body.is_in_group("enemy"):
        #If it is, reduce the player's health by 10
        Global.player_health -= 10
    pass # Replace with function body.

```

It may seem counter-intuitive to not have the player's health variable in the Player script itself. However, by adding it to the Autoload instead we can access it from *any* script in our game and it will *persist* if we change scenes (such as if we change levels) or if the Player is reloaded. Can you think of any other *data* you may want to share between scenes?

Accessing a method()

Let's consider that we have 3 different health collectibles, each with its own `healing_value` property. Let's also consider that these collectibles are `RigidBody2Ds`, so they will be caught by our Player's `Area2D` as well. First, we'll modify our `body_entered` signal function to check if the entering body is the group "healing" if it is not in the group "enemy":

```

func _on_area_2d_body_entered(body: Node2D) -> void:
    # First, check if the entering PhysicsBody is in the "enemy" group
    if body.is_in_group("enemy"):
        #If it is, reduce the player's health by 10
        Global.player_health -= 10
    elif body.is_in_group("healing"):

```

```
pass
pass # Replace with function body.
```

Next, let's call the `heal()` method on our "Global" Autoload. We do this through dot notation as well:

```
func _on_area_2d_body_entered(body: Node2D) -> void:
    # First, check if the entering PhysicsBody is in the "enemy" group
    if body.is_in_group("enemy"):
        #If it is, reduce the player's health by 10
        Global.player_health -= 10
    elif body.is_in_group("healing"):
        Global.heal()
    pass
pass # Replace with function body.
```

You'll remember that we added an *argument* for our `heal()` method(), an *argument* is data that we feed *into* a method() or function. In this case, we want to send *how much* to heal by into the `heal()` method(). Remember, we have given the healing collectibles a property called `healing_value` that we want to send into the `heal()` method(), we can access the `healing_value` property through dot notation as well:

```
func _on_area_2d_body_entered(body: Node2D) -> void:
    # First, check if the entering PhysicsBody is in the "enemy" group
    if body.is_in_group("enemy"):
        #If it is, reduce the player's health by 10
        Global.player_health -= 10
    elif body.is_in_group("healing"):
        Global.heal(body.healing_value)
    pass
pass # Replace with function body.
```

Now it's your turn!

In your Basket Catch project do the following tasks:

1. Create a new Autoload script named "Global"
2. Add a property to the "Global" Autoload called "points"
3. Add a script to each of your falling objects and add a variable called "point_value"
 1. Assign a different value to each falling object's "point_value" variable
4. Modify the "Player" script's `on_area_2d_body_entered` signal function to include the following:

1. Add a conditional statement that only runs if the entering body is in the "collectible" group.
2. Increase the value of the "Global" script's "points" property by the value in the falling object's "point_value" property.
3. Print the value of the "Global" script's "points" property to the console